# Лабораторна робота №10. AI-агент для JavaScript debugging: аналіз Console errors, Stack Trace, DOM та Event Loop

**Термін здачі: 21.10.2026**

## 1. Мета роботи

Навчитися системно знаходити першопричину дефектів у JavaScript-застосунку за допомогою Chrome DevTools: від повідомлення в Console та Stack Trace — до фактичного стану DOM і порядку виконання коду в Event Loop. Навчитися використовувати AI-агента як партнера з debugging, який допомагає формулювати й відкидати гіпотези, а не як генератор виправлень, і перевіряти кожну його рекомендацію доказами з браузера.

**Очікуваний результат:** для кожного знайденого дефекту студент може показати ланцюжок **відтворення** → **докази** → **гіпотеза** → **перевірка** → **мінімальний patch** → **повторний тест** і пояснити, чому виправлення усуває саме першопричину, а не симптом.

## 2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Розрізняти `ReferenceError`, `TypeError`, `SyntaxError`, необроблене відхилення Promise (`Uncaught (in promise)`) та логічні дефекти, які взагалі не створюють exception.
- Читати Stack Trace, зокрема його асинхронну частину, і відрізняти місце, де помилка **виявилася**, від місця, де закладено її **причину**.
- Користуватися панеллю Sources: breakpoints, *Pause on exceptions*, Scope, Watch, покрокове виконання.
- Досліджувати фактичний DOM через Elements і Console: `$0`, `$()`, `$$()`, `getEventListeners()`, вкладку Event Listeners, DOM breakpoints.
- Пояснювати порядок виконання коду через Call Stack, task queue, microtask queue та Event Loop — зокрема те, що async-функція виконується синхронно до першого `await`.

- Формулювати prompt для AI-агента з доказами та обмеженнями, а не з проханням «виправ».
- Розпізнавати виправлення, яке маскує симптом (`try/catch`, `?.`, `setTimeout` «із запасом»), і пояснювати, чому воно неприйнятне.
- Оформлювати кожне виправлення окремим мінімальним комітом, який можна відтворити й перевірити.

# 3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4) і провести експерименти в браузері.
2. Завантажити наданий starter-проєкт Courtly Booking, запустити його локально й зафіксувати вихідний стан у Git.
3. Знайти, дослідити та виправити щонайменше по одному дефекту в кожному з трьох сценаріїв: A — Console і Stack Trace, B — DOM, C — Event Loop (§5).
4. Провести експеримент із порядком виконання обробників подій (§5.5, C.2).
5. Провести регресійну перевірку всього застосунку (§5.6).
6. Оформити звіт (§6) і усно захистити роботу (§7).

> **Важливо!** Starter-проєкт — це інша сторінка того самого сервісу Courtly з ЛР-7: каталог майданчиків із пошуком і бронюванням. Він містить навмисно допущені дефекти всіх трьох типів; їх кількість і місце не повідомляються. Власний проєкт ЛР-7 для цієї роботи не підходить: JavaScript у ньому — лише mobile menu, а дефекти мають бути однаковими для всіх, щоб результати можна було порівняти.

# 4. Дослідницькі завдання

*Перед виконанням практичної частини опрацюйте джерела й проведіть експерименти. У звіт додайте короткі висновки (3–5 речень на підрозділ) і результати експериментів. На захисті ці питання доведеться пояснювати без допомоги AI.*

## 4.1. Console і типи помилок

**Опрацювати:**

1. JavaScript error reference — MDN
2. Console overview — Chrome DevTools
3. Console features reference — Chrome DevTools

**Дослідити:** чим відрізняються `ReferenceError`, `TypeError` і `SyntaxError`; чому `SyntaxError` під час розбору скрипта не дає виконатися жодному його рядку, а

необроблений `TypeError` перериває лише поточне завдання — решта сторінки продовжує працювати; що означає `Uncaught (in promise)`; які дефекти взагалі не потрапляють у Console. Як працюють рівні повідомлень і фільтри, навіщо **Preserve log**.

### Експеримент

У Console виконайте по одному рядку, що спричиняє `ReferenceError`, `TypeError` і `SyntaxError` під час розбору коду, а також `Promise.reject(new Error('test'))`. Порівняйте повідомлення: що спільного, чим відрізняються, яке з них не має Stack Trace і чому. Потім виконайте `JSON.parse('{')`: це теж `SyntaxError`, але чи має він Stack Trace — і чому інакше, ніж у попередньому випадку? Окремо поясніть, чому **перша** помилка в Console зазвичай важливіша за всі наступні.

## 4.2. Stack Trace і панель Sources

**Опрацювати:**

4. Error.prototype.stack — MDN
5. Debug JavaScript — Chrome DevTools
6. Pause your code with breakpoints — Chrome DevTools
7. Debugging asynchronous JavaScript with Chrome DevTools

**Дослідити:** як читати стек згори донизу; що означають ім'я функції, файл, рядок і колонка; що таке асинхронна частина стеку (рядки з позначкою `(async)`) і звідки вона береться. Типи breakpoints: line-of-code, conditional, logpoint, DOM change, event listener, exception (*Pause on uncaught / caught exceptions*). Що показують Scope, Watch і Call Stack, коли виконання зупинене.

**Ключове питання:** верхній рядок стеку — це місце, де помилка **виявилася**. Чи завжди там і її **причина**? Наведіть приклад, коли ні.

### Експеримент

Напишіть ланцюжок функцій `a() → b() → c()`, де `c()` звертається до властивості `undefined`, і викличте `a()` через `setTimeout`. Знайдіть у стеку в Console його асинхронну частину: який рядок її позначає і що він означає? Потім повторіть експеримент з увімкненим *Pause on uncaught exceptions*: що Sources показує понад те, що є в Console?

## 4.3. DOM у DevTools

**Опрацювати:**

8. View and change the DOM — Chrome DevTools
9. Console Utilities API reference — Chrome DevTools
10. Document: querySelector() — MDN

11. NodeList — MDN
12. Event bubbling — MDN

**Дослідити:** чим DOM у панелі Elements відрізняється від HTML-файлу та від *View page source*, коли сторінку змінює JavaScript; `$0`, `$()`, `$$()`, `getEventListeners()`, `monitorEvents()`; вкладку **Event Listeners**; чим відрізняються `querySelector`, `querySelectorAll` і `getElementById` та що повертає кожен, якщо нічого не знайдено; live і static колекції; event delegation. Обробник події прив'язаний до конкретного вузла DOM, а не до селектора, — що з цього випливає?

### Експеримент

На сторінці starter-проєкту виділіть будь-яку кнопку в Elements і перевірте її обробники двома способами: вкладкою Event Listeners і `getEventListeners($0)` у Console. Чи збігаються результати?

## 4.4. Event Loop

**Опрацювати:**

13. JavaScript execution model — MDN
14. Using microtasks in JavaScript with queueMicrotask() — MDN
15. In depth: Microtasks and the JavaScript runtime environment — MDN
16. Tasks, microtasks, queues and schedules — Jake Archibald
17. async function — MDN

**Дослідити:** Call Stack; роль Web APIs (таймери, мережа, події); task (macrotask) і microtask queue; коли саме виконуються microtasks і чому їхня черга спорожнюється повністю; де в циклі відбувається рендеринг; коли виконується callback `setTimeout(…, 0)` і чому «0 мс» не означає «одразу»; коли виконується callback, переданий у `.then()`, і продовження після `await`; чому async-функція виконується синхронно до першого `await` і що вона повертає в цей момент.

> Відповідь «Promise виконується швидше за `setTimeout`» не зараховується. Потрібно пояснити, **у яку чергу** потрапляє кожен callback, **що** його туди ставить і **коли** Event Loop передає його в Call Stack.

### Експеримент

Спочатку запишіть прогноз, потім перевірте в Console:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
queueMicrotask(() => console.log('4'));
(async () => {
  console.log('5');
  await null;
  console.log('6');
})();
console.log('7');
```

Якщо прогноз не збігся з результатом — поясніть, у якому саме місці.

## 4.5. AI-assisted debugging

**Дослідити**, у чому AI-агент реально допомагає під час debugging: аналіз повідомлення та стеку, кілька конкурентних гіпотез, пошук пов'язаного коду, мінімальний patch, перевірки після виправлення. І де він помиляється:

- виправляє симптом, а не причину;
- змінює зайвий код;
- «бачить» причини, яких немає ні в стеку, ні в коді;
- пропонує `try/catch`, optional chaining (`?.`) або значення за замовчуванням, які приховують помилку, але не виправляють її;
- пропонує `setTimeout` «із запасом» замість виправлення порядку виконання;
- змінює поведінку застосунку замість відновлення очікуваної.

# 5. Практична частина: AI-assisted debugging

**Courtly Booking** — каталог майданчиків сервісу Courtly: пошук за містом і видом спорту, вибір дати, картки майданчиків, діалог бронювання та список бронювань. Як застосунок **має** працювати, описано в ТЗ: дефектом вважається будь-яка розбіжність з ним.

```
     Відтворення
          ↓
Спостереження та докази
(Console, Stack Trace, Elements, Sources)
          ↓
  Власні гіпотези
          ↓
AI-агент: гіпотези та перевірки
          ↓
Перевірка в DevTools
          ↓
 Мінімальне виправлення
          ↓
  Повторне відтворення
          ↓
Регресійна перевірка
```

## 5.1. Підготовка

1. Розпакуйте архів в **окремий репозиторій** або в теку `lab-10/` спільного репозиторію з лабораторними.
2. Зробіть перший коміт **без змін** — це точка відліку для всіх diff у звіті й на захисті.
3. Запустіть проєкт через локальний сервер (Live Server, `npx serve` тощо). Через `file://` застосунок не запуститься: ES-модулі та `fetch()` локального JSON браузер із такої адреси блокує.
4. Відкрийте сторінку в Chrome або Edge, DevTools → Console. Увімкніть **Preserve log**. Перед кожним сценарієм очищайте Console.
5. Не змінюйте код, доки дефект не відтворено, а докази не зафіксовано.

## 5.2. Правила роботи з AI-агентом

Для кожного сценарію передайте AI-агенту щонайменше: очікувану та фактичну поведінку, кроки відтворення, повідомлення Console, Stack Trace (або його релевантну частину), фрагмент коду, а для DOM-дефекту — фактичний стан елемента з Elements.

Перший prompt — **без права змінювати код**. Наприклад:

> Проаналізуй цю помилку як senior JavaScript debugger. Назви 2–3 можливі першопричини, спираючись лише на наведені Console, Stack Trace і код. Для кожної запропонуй перевірку в DevTools, яка її підтвердить або відкине. Код не змінюй.

Мінімальний patch просіть лише після того, як **самі** підтвердили причину в DevTools. Якщо агент працює в IDE, для першого кроку ввімкніть режим без редагування (ask / plan). Якщо агент має доступ до браузера (наприклад, через MCP-сервер DevTools), це дозволено, але кожен доказ у звіті відтворіть і зафіксуйте у власному DevTools.

У звіті вкажіть назву та версію агента, модель і дату роботи.

## 5.3. Сценарій A — Console і Stack Trace

Знайдіть дефект, через який під час дії користувача в Console з'являється необроблена помилка. Якщо таких помилок кілька, для звіту оберіть ту, в якої місце виявлення і місце причини **різні**: саме для неї мають сенс кроки 4 і 6. Решту теж можна виправити — кожну окремим комітом.

1. Відтворіть помилку й запишіть точні кроки.
2. Зафіксуйте повідомлення Console повністю, разом із типом помилки.
3. Розгорніть Stack Trace. Для кожного кадру запишіть функцію, файл і рядок. Позначте, де починається асинхронна частина стеку, якщо вона є.
4. Зупиніть виконання в місці помилки — breakpoint або *Pause on uncaught exceptions*. Через Scope / Watch з'ясуйте, яке значення спричинило помилку і **звідки воно прийшло**.
5. Сформулюйте щонайменше **дві гіпотези** — власну та від AI — і перевірте кожну. Відкинута гіпотеза — теж результат: запишіть, яким доказом її відкинуто.
6. Внесіть мінімальне виправлення там, де закладено причину, а не там, де помилка виявилася.
7. Повторіть сценарій. Помилка, яка зникла з Console, ще не означає виправлений дефект: переконайтеся, що застосунок показує саме те, що описано в ТЗ.

**Обов'язковий результат** — зв'язка:

```
Console → Stack Trace → значення в точці зупинки → звідки воно прийшло
        → гіпотези → patch → повторний тест
```

## 5.4. Сценарій B — DOM

Знайдіть дефект, за якого **в Console немає жодної помилки**, але інтерфейс поводиться не так, як описано в ТЗ.

1. Опишіть очікувану та фактичну поведінку.
2. Знайдіть елемент в Elements. Перевірте `id`, `class`, атрибути, вміст і вкладку **Event Listeners**.
3. Перевірте в Console, що саме повертає запит, який використовує код (`$()`, `$$()`, `document.querySelector(…)`), і чи має елемент обробники (`getEventListeners($0)`).
4. Покажіть розбіжність: чого очікує код і що фактично є в DOM у момент виконання.
5. Якщо незрозуміло, **хто і коли** змінює елемент, поставте DOM breakpoint (*Break on → subtree modifications / attribute modifications*).
6. Проаналізуйте проблему з AI-агентом, виправте, перевірте.

Окремо поясніть, чому перевірка в DevTools дає сильніший доказ, ніж читання HTML-файлу.

## 5.5. Сценарій C — Event Loop

**C.1. Дефект.** Знайдіть дефект, причина якого — порядок виконання коду: щось відбувається раніше або пізніше, ніж очікує автор коду.

1. Відтворіть дефект. Додайте тимчасові `console.log` або logpoints у Sources у ключові місця й запишіть фактичний порядок.
2. Поясніть цей порядок: що виконується синхронно, що потрапляє в task queue, що — у microtask queue, і коли Event Loop передає кожен callback у Call Stack.
3. Побудуйте схему саме для цього дефекту, а не загальну:

```
Call Stack → Web API (таймер / мережа / подія) → task queue → Event Loop →
Call Stack
Promise / await ──────────────────────────→ microtask queue
                    (спорожнюється після кожного task, до рендерингу)
```

4. Виправляйте **порядок, а не час**. Рішення на кшталт «обгорнути в `setTimeout` на 500 мс» не приймається, навіть якщо після нього дефект «зникає». Поясніть, чому.

**C.2. Експеримент: два обробники кліку.** Вставте в Console на сторінці starter-проєкту:

```
const btn = document.createElement('button');
btn.textContent = 'Event Loop demo';
document.body.prepend(btn);

btn.addEventListener('click', () => {
  console.log('listener 1');
  setTimeout(() => console.log('timeout 1'), 0);
  Promise.resolve().then(() => console.log('promise 1'));
});

btn.addEventListener('click', () => {
  console.log('listener 2');
  queueMicrotask(() => console.log('microtask 2'));
});
```

1. **До запуску** запишіть прогноз порядку виводу для двох випадків: (а) клік мишею по кнопці; (б) виклик у Console `btn.click(); console.log('after click()');`.
2. Попросіть AI-агента дати прогноз для обох випадків, **не показуючи** йому результату.
3. Виконайте обидва варіанти й порівняйте з обома прогнозами.
4. Поясніть, чому порядок у випадках (а) і (б) різний. Підказка: коли саме Call Stack порожній?

## 5.6. Регресійна перевірка

Після всіх виправлень пройдіть **усю** очікувану поведінку з ТЗ (§3), а не лише ті місця, які ви змінювали: виправлення в одному місці часто ламає інше. Зафіксуйте результат у таблиці:

| Поведінка з ТЗ | До виправлень | Після |
|---|---|---|
| 3.1 Header і mobile menu | … | … |
| 3.2 Початковий стан | … | … |
| … | … | … |

Дефекти, які ви знайшли, але не виправили, теж запишіть — з поясненням.

## 5.7. Коміти

Кожне виправлення — **окремий коміт**, повідомлення якого називає сценарій і причину, наприклад `fix(A): <що було причиною і що змінено>`. Diff коміту — це ваш мінімальний patch: на захисті він покаже, чи не змінено зайвого. Тимчасові `console.log` у фінальний код не потрапляють.

# 6. Вимоги до звіту

**Тема листа:**

```
[Веб] ЛР-10: AI-агент для JavaScript debugging: аналіз Console errors, Stack Trace, DOM та Event Loop
```

У листі вкажіть репозиторій і адресу опублікованого проєкту — один із двох рядків, GitHub Pages або Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. висновки дослідницьких завдань (§4) і результати експериментів;
2. назву та версію AI-агента, модель, дату роботи;
3. для сценаріїв **A** і **B**: очікувану та фактичну поведінку, кроки відтворення; Console до виправлення; Stack Trace (A) або стан DOM і результати запитів у Console (B); гіпотези та спосіб перевірки кожної; prompt і стислу відповідь AI; посилання на коміт із виправленням; Console і поведінку після виправлення;
4. для сценарію **C**: фактичний порядок логів, пояснення, схему, prompt і стислу відповідь AI, посилання на коміт; для експерименту C.2 — ваш прогноз, прогноз AI та фактичний результат для обох випадків;
5. таблицю регресійної перевірки (§5.6);
6. підсумок: у яких ситуаціях AI-агент реально скоротив час debugging, а в яких його відповідь потребувала перевірки в DevTools; **один конкретний приклад** пропозиції AI, яка замаскувала б першопричину, — з поясненням, що саме вона приховала б. Якщо агент щоразу одразу знаходив першопричину, попросіть його виправити помилку лише у файлі, де вона виявилася, і проаналізуйте цю пропозицію.

Скриншоти мають показувати **доказ** — повідомлення, стек, значення в Scope, обробники елемента, — а не просто відкритий DevTools.

> **Важливо!** Звіт не замінює результату: здається **і** звіт, **і** репозиторій з окремими комітами виправлень, **і** опублікована виправлена версія.

# 7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Навіть якщо гіпотези чи виправлення запропонував AI-агент, ви повинні вміти пояснити кожну

зміну, показати, як її перевірено, і відповісти на питання з демонстрацією екрана.

Викладач може відкотити ваш patch (`git checkout` першого коміту) і попросити повторити debugging без AI або змінити умову сценарію.

**Орієнтовні питання:**

*Console і Stack Trace*

- Відтворіть дефект зі сценарію A на невиправленій версії. Прочитайте Stack Trace без допомоги AI: де помилка виявилася і де її причина?
- Зупиніть виконання в місці помилки й покажіть у Scope значення, яке її спричинило.
- Чим `TypeError` відрізняється від `ReferenceError`? Що означає `Uncaught (in promise)`?

*DOM*

- Покажіть у DevTools, де саме була розбіжність між кодом і DOM. Які обробники має цей елемент зараз?
- Чим `querySelector` відрізняється від `querySelectorAll` і `getElementById`? Що повертає кожен, якщо нічого не знайдено?

*Event Loop*

- Поясніть порядок виконання в сценарії C за вашою схемою. Чому `setTimeout` із затримкою не є виправленням?
- Чому в експерименті C.2 порядок відрізняється для кліку мишею і `btn.click()`?
- Що повертає async-функція в момент першого `await` і що відбувається далі?

*AI*

- Яку рекомендацію AI ви відхилили і чому? Що саме вона приховала б?
- Як ви перевірили, що прийняте виправлення AI усуває причину, а не симптом?

# 8. Оцінювання

За роботу — **3 бали**, по одному за дослідження, практичну частину й захист, і **1 додатковий бал**.

| Складник | Бали | Як оцінюється |
|---|---|---|
| Дослідження (§4) | 1 | по 0,25 за кожен підрозділ §4.1–§4.4: висновок на 3–5 речень і результат експерименту з поясненням (п. 1 звіту) |
| Практична частина (§5) | 1 | по 0,25 за кожен сценарій A, B і C: дефект відтворено, у звіті є всі докази з пп. 3–4, і окремий коміт усуває першопричину, а не симптом (для C — ще й експеримент C.2). Ще 0,25 — таблиця регресійної перевірки (п. 5), опублікована виправлена версія і приклад пропозиції AI, яка замаскувала б першопричину (п. 6) |

| Складник | Бали | Як оцінюється |
|---|---|---|
| Захист (§7) | 1 | debugging наживо, без AI: на невиправленій версії відтворити дефект одного зі сценаріїв і показати в DevTools його причину — до 0,5; два питання з різних груп §7 — по 0,25 за повну відповідь |
| Додатковий бал | +1 | одна з умов: термін здачі до **21.10.2026** АБО робота на парі |

> **Важливо!** Без захисту за дослідження й практичну частину разом — не більше **1 бала**: захист підтверджує, що ви розумієте зроблене, а не лише отримали результат за допомогою AI. Додатковий бал зараховується й без захисту.